

Homework 01

Daniel Nkunga

dnkunga@uark.edu

100202653

1. Describe the purpose of the homework and what you did / what you implemented.

This assignment was to implement both a linked list and queue data structures.

A linked list is a linear data structure where each piece of data, referred to as a node, points to the next piece of data in the sequence. To traverse a linked list, the head must be known, and each consecutive node must be visited to find the element desired to be accessed. For this assignment, I also implemented a doubly ended linked list [undergrad, not required] where each node also points to the previous node.

A queue is a linear data structure where data is interfaced following a first-in-first-out pattern. To interface with a queue, the top element of the queue can be popped, or a new element can be pushed to the back of the queue.

Describe your approach and explain how you solved the problem (the key ideas for implementing each required function).

To implement the deconstructor for the linked list, I needed to program the basic lines of code that allow me to traverse the linked list. For me, that meant having two nodes keeping track of the current and next node, deleting the current node, and constantly reassigning next to current so long as there was a next node to continue deleting.

To implement the insert function for the linked list, I used the current-next process until I found there was no next note, then I added the new node there.

To implement the find function, I also implemented the current-next process until I found the node I was looking for, or until there were no next elements for me to traverse.

Implementing the remove function immediately started out simple because the first check was to see if the node trying to be removed existed. Though I could've used continued using the current-next process, it was the prefect case to use the previously implemented find function. If the node deleted didn't exist, I returned the root. If the node existed, I would delete the node. This is also the part I had the most fear of implementing the doubly linked list logistically. If the logic failed here, the entire class would fall apart any

time one element got deleted due to strand pointing to oblivion.

The size function was a simple current-next process with a counter built into it.

To implement the destructor function, I used a variation of the current-next process I used often in the linked list. In this case, where there was a next element, I'd save it, delete the current element (always pointing at the head) then assign the next element to be the head and iterate. The only other odd thing I missed initially was assigning the head and tail to be null afterwards.

To implement the empty function on the queue, I implemented a one-line function that checked if both the head and tail were null.

Implementing the pop function came by storing the value of the head, deleting the head, and assigning the next value in line to be the head. The stored value would be returned and, if the queue was now empty, the tail would also turn to a null pointer.

Since the pointer for the tail is maintained, pushing a new element to the back and assigning it to be the new tail. If the queue was empty, it would also be the head.

2. Discuss any bugs or problems you encountered and how you fixed them.

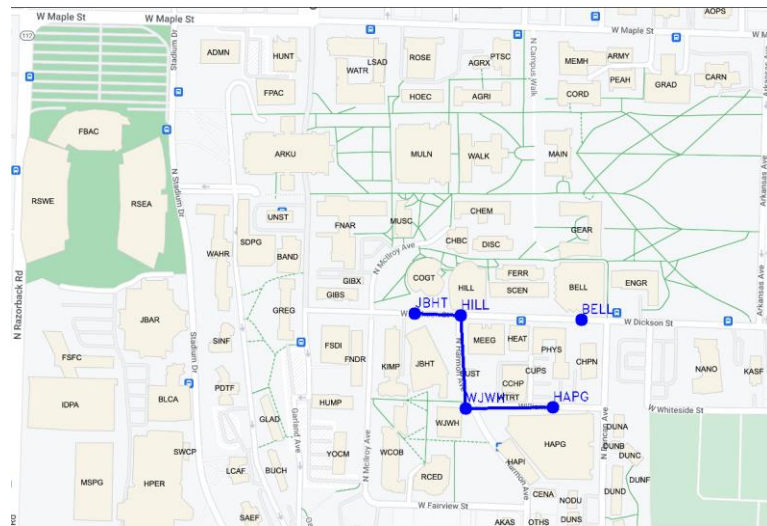
The initial major issue I had was when using the make file. I had never seen the "-I" flag before and initially thought my program was not compiling because of both the unimplemented functions and the missing file paths to the header files. It took me fully implementing one of the programs while forgetting to make the header file path explicit to realize and explore what was going on in the make file.

My insert function has seemingly redundant code that I tried removing the day after I wrote, but it stopped my program from compiling due to seemingly a "missing return statement." Though I do see the program's concerns, in reality there should never be a circumstance where that final piece of code runs. If the list is empty, its caught by the first if-statement. If its, not, it'll be caught by one of the two statements in the while-loop. Technically its neither a problem nor but, its just annoying.

If I were to develop the linked list further, I think it would be useful to keep track of both the size of the list and the last element. Keeping track of the size would allow for a few more array interactions that could prove niche (niche because you wouldn't know the

data being stored at each index) but interesting. Keeping track of the last element could allow for backwards traversal to become an even more viable for of operating as, if an element is known to be at the end or you just want to preform operations at the last element, it could be done without traversing the entire list.

3. Include screenshots or images of your results.



```
• [daniel_n@akihayakawa Hw01]$ ./bin/main
10 is in the linked list
20 is not in the linked list
100 is in the linked list
The linked list has 2 nodes
100 is not in the linked list
The linked list has 1 node
Your linked list implementation is correct

Queue is not empty
Remove first node in queue successfully
Queue is empty
Your queue implementation is correct

Path from 0 to 5: 0 1 2 4 5
Your linked list and queue implementation is correct

Path from JBHT to HAPG: JBHT -> HILL -> WJWH -> HAPG

QSettings::value: Empty key passed
Press ENTER on the image window to continue...

QThreadStorage: entry 8 destroyed before end of thread 0x5573053e3dc0
QThreadStorage: entry 7 destroyed before end of thread 0x5573053e3dc0
QThreadStorage: entry 2 destroyed before end of thread 0x5573053e3dc0
QThreadStorage: entry 1 destroyed before end of thread 0x5573053e3dc0
○ [daniel_n@akihayakawa Hw01]$
```